

Week 12 — Summary

Phase 3: Docker & Clean Architecture · ML Engineer Journey

Phase	Phase 3 — Docker & Clean Architecture
Week	12 of 24
Days covered	W12D01 · W12D03 · W12D04 · W12D05 · W12D06
Status	COMPLETE ✓
GitHub	github.com/lolivampire/ML-Project

1. Week 12 at a Glance

Week 12 adalah minggu validasi dan konsolidasi. Tidak ada konsep baru yang diperkenalkan — sebaliknya, semua yang dibangun di Phase 3 (Week 9–12) digabungkan menjadi satu sistem yang koheren: sebuah containerized ML API yang production-grade. Fokus minggu ini adalah membuktikan bahwa setiap layer bekerja tidak hanya secara terpisah, tetapi juga bersama-sama sebagai satu kesatuan.

Hari	Topik	Output Utama
W12D01	Multi-Stage Docker Build	Dockerfile dua stage, image ~180MB
W12D03	Health Check & Graceful Shutdown	/health + /ready endpoint, urutan shutdown
W12D04	Buffer — Testing Coverage	pytest semua layer: unit + integration
W12D05	Final Documentation (README)	README 7-section production-grade
W12D06	Final Review & Week Summary	Integration checklist, commit final, PDF ini

2. Multi-Stage Docker Build

Analogi: bayangkan kamu membangun meja. Kamu butuh gergaji, palu, amplas, dan bor — tapi ketika meja selesai dan dikirim ke rumah pelanggan, kamu tidak ikutkan semua alat tersebut dalam pengiriman. Multi-stage build bekerja dengan cara yang sama: semua build tools digunakan di stage pertama, tapi hanya artifact yang benar-benar dibutuhkan saat runtime yang dibawa ke final image.

Struktur Dockerfile dua stage:

FROM python:3.11 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user -r requirements.txt # install semua dep
FROM python:3.11-slim AS production # image ringan
WORKDIR /app
COPY --from=builder /root/.local /root/.local # ambil dep saja
COPY . . # copy source

```
CMD ["uvicorn", "app.main:app"]
```

Stage	Base Image	Ukuran Tipikal	Isi
builder	python:3.11	~900 MB	GCC, build tools, pip cache
production	python:3.11-slim	~180 MB	Runtime only, no build tools

Penghematan: ~80% ukuran image — kritis untuk deployment speed dan security surface.

Prinsip penting yang harus diingat:

- Layer caching: COPY requirements.txt sebelum COPY . . — agar pip tidak re-run jika code berubah tapi dependency tidak.
- Jangan copy .git, __pycache__, venv ke dalam image — gunakan .dockerignore.
- COPY --from=builder hanya copy apa yang diperlukan, bukan seluruh builder stage.

3. Health Check & Graceful Shutdown

Analogi: bayangkan sebuah restoran. /health adalah pertanyaan 'Apakah lampu masih menyala?' — jawabannya selalu ya selama listrik ada. /ready adalah pertanyaan 'Apakah dapur siap menerima pesanan?' — jawabannya tidak jika chef belum datang atau bahan habis. Kedua endpoint ini memberikan informasi yang berbeda kepada orchestrator (Kubernetes, Railway, dll).

Endpoint	Kondisi Return 200	Kondisi Return 503	Tujuan
/health	Selalu (proses hidup)	Tidak pernah	Liveness check — restart jika mati
/ready	Semua dependency OK	Redis / DB mati	Readiness — stop traffic jika tidak siap

Urutan Graceful Shutdown — WAJIB dihafalkan:

Urutan ini bukan sembarang urutan. Setiap langkah memiliki alasan spesifik:

Urutan	Aksi	Kenapa harus di sini
1 (pertama)	ready = False	Stop traffic baru masuk SEBELUM apapun dibongkar
2	Tunggu drain	Selesaikan request yang sudah terlanjur masuk
3	Close connections	Tutup Redis, DB, file handles dengan rapi
4 (terakhir)	Exit / return	Proses benar-benar berhenti

Jika ready=False tidak dilakukan PERTAMA, load balancer akan terus kirim request baru saat kamu sedang menutup koneksi — menghasilkan partial responses atau errors.

4. Testing Coverage — Pytest Semua Layer

Testing bukan sekadar 'pastikan tidak error'. Testing adalah kontrak: kamu mendefinisikan apa yang seharusnya terjadi, dan pytest memverifikasinya setiap kali kode berubah. Di arsitektur berlapis, setiap layer dites secara independen — ini yang disebut unit test. Kemudian endpoint dites end-to-end — ini

integration test.

Layer	Tipe Test	Yang Ditest	Tool
Schema (Pydantic)	Unit	Validasi input: tipe, range, required field	pytest + raises
Service	Unit	Business logic, transformasi data	pytest + mock
Repository	Unit	Model load, serialization	pytest + mock joblib
Endpoint (/predict)	Integration	Request → response full flow	TestClient FastAPI
Health/Ready	Integration	Status codes, dependency mock	TestClient FastAPI

Pola test yang harus selalu diikuti:

```
def test_predict_valid_input():
    response = client.post('/predict', json={'feature': 1.5})
    assert response.status_code == 200 # cek HTTP code
    assert 'prediction' in response.json() # cek struktur response

def test_predict_invalid_input():
    response = client.post('/predict', json={}) # tanpa required field
    assert response.status_code == 422 # Pydantic validation error
```

Penting: test SELALU harus mencakup happy path (input valid) dan sad path (input invalid / dependency mati). Test yang hanya menguji happy path memberikan false confidence.

5. Production README — 7 Section Wajib

README production-grade adalah manual servis, bukan brosur marketing. Tujuannya satu: seorang engineer baru bisa clone repo, setup environment, dan menjalankan API — tanpa harus bertanya kepada siapapun. Jika mereka harus bertanya, README gagal tugasnya.

Section	Isi Wajib	Audiens
1. Header	Badge status, nama project, deskripsi 1 kalimat	Semua
2. About	Masalah yang diselesaikan, tech stack, context bisnis	PM / Engineer baru
3. Architecture	Diagram layer, flow request, komponen utama	Engineer
4. Folder Structure	Tree direktori + penjelasan tiap folder	Engineer
5. Quick Start	Prerequisites → clone → install → run (step by step)	Engineer baru
6. API Reference	Endpoint, method, request body, response schema, error codes	API consumer
7. Testing	Cara run test + output pytest yang diharapkan (social proof)	Engineer / Reviewer

Social proof penting: tempelkan output pytest di README. Reviewer tidak perlu clone repo untuk tahu bahwa semua test hijau — mereka bisa lihat langsung di README.

6. Docker Compose — Multi-Service Orchestration

docker-compose.yml adalah deklarasi lengkap semua yang dibutuhkan untuk menjalankan sistem: service apa saja, bagaimana mereka terhubung, port mana yang di-expose, volume mana yang dipersist, dan environment variable apa yang diinjeksi.

```
services:
  api:
    build: .
    ports: ['8000:8000']
    environment:
      - REDIS_URL=redis://redis:6379
    depends_on:
      redis:
        condition: service_healthy # tunggu redis siap
    redis:
      image: redis:7-alpine
      volumes: [redis_data:/data] # data persist di-restart
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 10s
```

depends_on dengan condition: service_healthy memastikan API tidak start sebelum Redis benar-benar ready — bukan hanya sebelum container Redis berjalan. Ini perbedaan yang kritis: container bisa running tapi service-nya belum siap menerima koneksi.

7. Final Integration Checklist

Checklist ini adalah standar minimum untuk menyatakan sebuah containerized service siap production. Gunakan sebagai referensi di project-project berikutnya.

Komponen	Verifikasi	Standar Lulus
Dockerfile	docker build berhasil	Image terbuild, size ~180MB
/health	curl localhost:8000/health	HTTP 200 selalu
/ready	Matikan Redis, hit /ready	HTTP 503 (container tetap hidup)
Graceful shutdown	docker stop → lihat logs	ready=False muncul pertama di log
pytest	pytest -v	0 failed, semua assertion pass
docker-compose	docker-compose up dari scratch	API + Redis berjalan, /predict response OK
README	Baca tanpa lihat kode	Bisa diikuti engineer baru tanpa tanya
GitHub	git log --oneline	Commit message informatif, tidak ada file missing

8. Konsep Kunci yang Harus Diingat

Multi-stage build

Builder stage untuk compile/install → production stage hanya runtime. Hasilnya image lebih kecil, attack surface lebih kecil, deploy lebih cepat.

Layer caching

Urutan instruksi Dockerfile penting. Yang jarang berubah (requirements.txt) harus di-COPY sebelum yang sering berubah (source code). Cache invalidity bersifat kaskade.

/health vs /ready

/health = 'apakah prosesnya hidup?' (liveness). /ready = 'apakah siap menerima traffic?' (readiness). Keduanya diperlukan untuk zero-downtime deployment.

Graceful shutdown order

ready=False PERTAMA → drain request aktif → close connections → exit. Membalik urutan ini = data loss atau request yang terpotong.

Separation of concerns

Router tidak tahu cara load model. Service tidak tahu cara routing. Repository tidak tahu cara validasi. Setiap layer punya satu tanggung jawab.

Social proof di README

Output pytest yang ditempel di README bukan vanity — ini adalah bukti yang bisa diverifikasi tanpa clone repo. Reviewer melihat ini sebagai tanda professionalism.

9. Phase 3 — Penutup

Phase 3 (Week 9–12) membahas Docker & Clean Architecture. Ini bukan sekadar belajar cara membuat Dockerfile — ini adalah shift mindset dari 'kode yang jalan di local' menjadi 'sistem yang bisa di-deploy, di-monitor, dan di-maintain oleh orang lain'.

Week	Fokus	Deliverable
Week 9	Docker fundamentals + image optimization	Dockerfile untuk Project 1
Week 10	Docker Compose + networking + volumes	Multi-service compose setup
Week 11	Layered architecture (Router/Service/Repo)	Clean API structure
Week 12	Testing + health/ready + final docs	Project 2 production-ready

Yang kamu bawa ke Phase 4 (Database & Observability): kemampuan untuk tidak hanya menulis kode yang benar, tapi juga membungkusnya dalam container yang observable, testable, dan deployable. Fondasi ini akan diperlukan saat menambahkan PostgreSQL, migration, structured logging dengan trace IDs, dan metrics collection.

ML Engineer Journey · Week 12 Summary

Phase 3 Complete — Next: Phase 4 Database & Observability